

IKE gRPC API Design Requirements

Table of Contents

1. Executive Summary	2
2. Background	2
2.1. The Strategic Tension	3
2.2. System Characteristics	3
3. Identity: PublicId vs. Nid	3
3.1. PublicId	3
3.2. Nid	4
3.3. The Constraint	4
3.4. Protobuf Representation of PublicId	4
4. The Coordinate System	4
4.1. STAMP Coordinate	4
4.2. Language Coordinate	5
4.3. Logic Coordinate	6
4.4. Navigation Coordinate	6
5. Recommended Architecture: Three-Tier Progressive Disclosure	7
5.1. Architecture Overview	7
5.2. Coordinate Handling by Tier	8
6. Tier 1: Graph RAG Ready	8
6.1. Design Philosophy	9
6.2. Service Requirements	9
6.2.1. GetDeviceView	9
6.2.2. GetSubgraph	9
6.2.3. SearchConcepts	10
6.2.4. GetGraphSnapshot	10
6.3. Tier 1 Coordinate Contract	10
7. Tier 2: Concept-Aware	11
7.1. Design Philosophy	11
7.2. Service Requirements	11
7.2.1. GetConceptWithSemantics	11
7.2.2. ResolveToText	11
7.2.3. GetSemanticsByPattern	12
7.2.4. QueryWithCoordinates	12
7.2.5. GetVersionHistory	12
7.2.6. CompareVersions	13
7.3. Tier 2 Coordinate Contract	13
7.4. Tier 2 Coordinate Override Structures	13

8. Tier 3: IKE Native	14
8.1. Design Philosophy	15
8.2. Service Requirements	15
8.2.1. GetConcept	15
8.2.2. GetSemantics (server streaming)	15
8.2.3. ListPatterns (server streaming)	15
8.2.4. GetPattern	16
8.2.5. ResolveWithCoordinates	16
8.2.6. QueryBySTAMP (server streaming)	16
8.2.7. StreamAllConcepts (server streaming)	17
8.3. Tier 3 Coordinate Contract	17
8.4. Tier 3 Coordinate Structures	17
9. Design Principles	18
9.1. The Dual-Return Pattern	18
9.2. Progressive Revelation of Complexity	18
9.3. Leverage Calculators, Don't Expose Them	19
9.4. FDA Compliance Awareness	19
10. Implementation Strategy	19
10.1. Phase 1: Launch with Tiers 1 and 2	19
10.2. Phase 2: Tier 3 and Community Patterns	19
10.3. Phase 3: Write Operations	19
11. Performance Considerations	20
11.1. Caching	20
11.2. Streaming vs. Unary	20
11.3. Batch Operations	20
12. Open Questions for Reviewer Feedback	20

1. Executive Summary

This document specifies requirements for a gRPC API layer over the IKE (Integrated Knowledge Exchange) versioned knowledge graph.^[1] The objective is to enable adoption by ML, AI, and data analytics developers while preserving the formal rigor of the underlying IKE architecture — particularly its coordinate system, STAMP versioning, and self-describing concept architecture.

The central architectural recommendation is a **three-tier progressive disclosure** model. Each tier serves a different developer persona and makes different decisions about how much of the coordinate system to expose. This document specifies the requirements for each tier and defines the contract between the gRPC API and the underlying Java coordinate records.

2. Background

2.1. The Strategic Tension

The IKE knowledge graph is a profoundly self-describing, formally rigorous system based on the HL7 TINKAR standard. Its coordinate system — which governs how concepts are resolved, which versions are visible, which language is preferred, and which logic profile is in effect — is both its greatest strength and its steepest adoption barrier.

The target developer population (ML engineers, RAG pipeline builders, analytics practitioners) wants immediate productivity. They should not need to understand the full coordinate system to get useful results. But the coordinate system must not be *hidden* — only *deferred* — because developers who need formal rigor, regulatory compliance, or multi-version analysis will eventually require explicit coordinate control.

2.2. System Characteristics

The IKE architecture implements:

- **STAMP versioning** — every component version is stamped with Status, Time, Author, Module, and Path
- **Concept-oriented self-description** — field datatypes, meanings, and purposes are all concepts
- **Semantic patterns** — extend concepts with structured, versioned information
- **Calculator-based caching** — idiomatic queries are served through cached calculators (e.g., `StampCalculatorWithCache`)
- **PublicId-based identity** — all components are identified by PublicIds (one or more UUIDs); internal nids are never exposed externally
- **Composite coordinates** — a ViewCoordinate combines stamp, language, logic, and navigation coordinates to fully specify a perspective on the knowledge graph

3. Identity: PublicId vs. Nid

A foundational constraint for the gRPC API is the distinction between **PublicId** and **nid**.

3.1. PublicId

A **PublicId** is the externally-visible identity of any IKE component (concept, semantic, pattern). Critically, a PublicId is **not** a single UUID — it is one or more UUIDs that all identify the same component. The **PublicId** interface provides `uuidCount()`, `asUuidArray()`, and `asUuidList()`.

Two PublicIds are considered equal if they share *any* UUID (set-intersection semantics). This supports scenarios where the same concept has been independently assigned UUIDs by different authoring organizations, and those assignments have subsequently been recognized as equivalent.

3.2. Nid

A nid (native identifier) is an `int` used internally by the IKE Java implementation for fast local lookup. Nids have no meaning outside the server process. They are not stable across database rebuilds, not portable across installations, and not suitable for wire transmission.

3.3. The Constraint

Nids must never appear in gRPC messages. All component references crossing the wire must use `PublicIds`. The server is responsible for translating between `PublicIds` and nids at the gRPC boundary.

This means:

- Protobuf messages representing component identity must carry one or more UUID values (a `PublicId`), not a single UUID string.
- The coordinate records, which internally use nids (e.g., `int classifierNid` in `LogicCoordinateRecord`), must be translated to `PublicId`-based representations at the gRPC layer.
- The `PublicIdWithString` pattern — a `PublicId` paired with a human-readable label — maps naturally to the dual-return pattern used at Tier 2.

3.4. Protobuf Representation of PublicId

The protobuf representation of a `PublicId` must support multiple UUIDs per component. The recommended approach is a message containing a repeated field of UUID values. The exact encoding of individual UUIDs (string representation vs. paired `int64` `msb/lbs`) is an implementation decision, but the message must be capable of carrying an arbitrary number of UUIDs for a single component identity.

4. The Coordinate System

The coordinate system is the heart of the API design problem. Every query against the IKE knowledge graph is made *from a perspective* defined by coordinates. The four coordinate types and their actual structures (derived from the Java record implementations) are specified below.

Understanding these structures is essential for reviewers evaluating whether the tier contracts are correct.

4.1. STAMP Coordinate

The STAMP coordinate governs *which versions are visible*. It is not a single STAMP tuple; it is a filter that selects versions matching specified criteria.

Table 1. StampCoordinateRecord — actual Java structure (nids translate to PublicIds at the gRPC boundary)

Field	Java Type	gRPC Type	Semantics
<code>allowedStates</code>	<code>StateSet</code>	StateSet (see [Open Questions])	Which status values pass the filter (e.g., ACTIVE only, or ACTIVE_AND_INACTIVE). This is a <i>set</i> , not a single value.
<code>stampPosition</code>	<code>StampPositionRecord</code>	time (int64 or Timestamp) + path (PublicId)	A (time, path) pair. Time is epoch millis or <code>Long.MAX_VALUE</code> for latest. Path is a concept identifying the development/release path.
<code>moduleNids</code>	<code>IntIdSet</code>	repeated PublicId	Set of module concepts to <i>include</i> . Empty means "allow any module."
<code>excludedModuleNids</code>	<code>IntIdSet</code>	repeated PublicId	Set of module concepts to <i>exclude</i> . Evaluated after inclusion.
<code>modulePriorityNidList</code>	<code>IntIdList</code>	repeated PublicId (ordered)	Ordered preference for module priority when multiple versions compete. Order matters.

Key design observations: the STAMP coordinate is a *filter*, not a *stamp*. It defines what you can see, not what something is. The inclusion/exclusion/priority pattern for modules is intentional — it supports federated content from multiple authoring organizations where a consuming site needs to prefer one module's content over another.

4.2. Language Coordinate

The language coordinate governs *how concepts are rendered as human-readable text*.

Table 2. *LanguageCoordinateRecord* — actual Java structure (*nids* translate to *PublicIds* at the gRPC boundary)

Field	Java Type	gRPC Type	Semantics
<code>languageConceptNid</code>	<code>int</code> (concept nid)	PublicId	The language (e.g., English, Spanish). A single concept.
<code>descriptionPatternNidList</code>	<code>IntIdList</code>	repeated PublicId (ordered)	Ordered list of description patterns to search. The system tries each pattern in order to find a description.
<code>descriptionTypePreferenceNidList</code>	<code>IntIdList</code>	repeated PublicId (ordered)	Ordered preference for description type (e.g., prefer "regular name" over "fully qualified name"). Order matters.
<code>dialectPatternPreferenceNidList</code>	<code>IntIdList</code>	repeated PublicId (ordered)	Ordered preference for dialect (e.g., prefer US English over GB English). Order matters.

Field	Java Type	gRPC Type	Semantics
<code>modulePreferenceNidListForLanguage</code>	<code>IntIdList</code>	repeated <code>PublicId</code> (ordered)	Module preference specific to language resolution. Allows preferring one organization's descriptions over another's.

Key design observation: every field except `languageConceptNid` is an *ordered preference list*. The resolution algorithm walks these lists in order to find the best available description. This is not a simple locale tag — it is a multi-axis preference cascade.

4.3. Logic Coordinate

The logic coordinate governs *which reasoning infrastructure is in effect*.

Table 3. *LogicCoordinateRecord* — actual Java structure (nids translate to `PublicIds` at the gRPC boundary)

Field	Java Type	gRPC Type	Semantics
<code>classifierNid</code>	<code>int</code> (concept nid)	<code>PublicId</code>	Which DL classifier to use (e.g., EL++ classifier).
<code>descriptionLogicProfileNid</code>	<code>int</code> (concept nid)	<code>PublicId</code>	Which description logic profile constrains authoring (e.g., EL++ profile).
<code>inferredAxiomsPatternNid</code>	<code>int</code> (pattern nid)	<code>PublicId</code>	Pattern identifying where inferred axioms are stored.
<code>statedAxiomsPatternNid</code>	<code>int</code> (pattern nid)	<code>PublicId</code>	Pattern identifying where stated axioms are stored.
<code>conceptMembershipPatternNid</code>	<code>int</code> (pattern nid)	<code>PublicId</code>	Pattern for concept membership semantics.
<code>statedNavigationPatternNid</code>	<code>int</code> (pattern nid)	<code>PublicId</code>	Pattern for the stated taxonomy navigation structure.
<code>inferredNavigationPatternNid</code>	<code>int</code> (pattern nid)	<code>PublicId</code>	Pattern for the inferred taxonomy navigation structure.
<code>rootNid</code>	<code>int</code> (concept nid)	<code>PublicId</code>	Root concept for the taxonomy.

Key design observation: the logic coordinate carries *eight* concept/pattern references, not just a reasoner choice. It wires together the full inference pipeline — from stated axioms through classification to inferred navigation. All eight are concept or pattern references, consistent with IKE's self-describing philosophy. Each nid becomes a `PublicId` at the gRPC boundary.

4.4. Navigation Coordinate

The navigation coordinate governs *how the taxonomy is traversed*.

Table 4. *NavigationCoordinateRecord* — actual Java structure (nids translate to `PublicIds` at the gRPC boundary)

Field	Java Type	gRPC Type	Semantics
<code>navigationPatternNids</code>	<code>IntIdSet</code>	repeated <code>PublicId</code>	Set of navigation patterns to include. Typically one of: stated navigation or inferred navigation.
<code>vertexStates</code>	<code>StateSet</code>	<code>StateSet</code> (see [Open Questions])	Which vertex states to include during traversal (e.g., <code>ACTIVE</code> only, or <code>ACTIVE_AND_INACTIVE</code>).
<code>sortVertices</code>	<code>boolean</code>	<code>bool</code>	Whether to sort child vertices in results.
<code>verticesSortPatternNidList</code>	<code>IntIdList</code>	repeated <code>PublicId</code> (ordered)	Ordered list of patterns to use as sort keys when <code>sortVertices</code> is true.

Key design observation: navigation is decoupled from logic. You can navigate an inferred hierarchy without re-running classification, because the navigation patterns are pre-computed. The vertex state filter is independent of the STAMP coordinate's allowed states — you might want to see active-only content (STAMP) but include inactive vertices in the tree structure (navigation) to preserve structural context.

5. Recommended Architecture: Three-Tier Progressive Disclosure

5.1. Architecture Overview

The API exposes three tiers via gRPC. Each tier serves a different developer persona and makes fundamentally different decisions about coordinate visibility.

Tier	Target Audience	Coordinate Handling	Time to Productivity
Tier 1: Graph RAG Ready	Data scientists, RAG engineers, prompt engineers	Coordinates are <i>invisible</i> . Server applies sensible defaults. Caller specifies at most a timestamp and a language tag.	Minutes to hours
Tier 2: Concept-Aware	Analytics engineers, knowledge graph practitioners	Coordinates are <i>optional with defaults</i> . Caller can override any coordinate axis. Server fills unspecified axes with defaults.	Hours to days
Tier 3: IKE Native	Integration developers, tooling builders, DL researchers	Coordinates are <i>required and explicit</i> . No defaults. Caller must specify the full coordinate context. Structures map faithfully to the Java records.	Days to weeks

The critical invariant across all three tiers: **every query executes against a fully-specified set of**

coordinates. The tiers differ only in *who is responsible for specifying them* — the server (Tier 1), a collaboration between caller and server (Tier 2), or the caller alone (Tier 3).

5.2. Coordinate Handling by Tier

The following table summarizes the coordinate contract at each tier. "Hidden" means the caller has no way to influence the coordinate. "Optional" means the caller can override it; the server fills in a default if omitted. "Required" means the caller must provide it.

Coordinate Axis	Tier 1	Tier 2	Tier 3
STAMP: allowed states	Hidden (ACTIVE only)	Optional (default: ACTIVE)	Required
STAMP: position (time)	Optional (default: latest)	Optional (default: latest)	Required
STAMP: position (path)	Hidden (server default path)	Optional (default: server default)	Required
STAMP: module inclusion/exclusion	Hidden (all modules)	Optional (default: all modules)	Required
STAMP: module priority	Hidden (server default priority)	Optional (default: server default)	Required
Language: language concept	Optional (default: en-US)	Optional (default: en-US)	Required
Language: description pattern preferences	Hidden	Optional	Required
Language: description type preferences	Hidden	Optional	Required
Language: dialect preferences	Hidden	Optional	Required
Language: module preferences for language	Hidden	Optional	Required
Logic: all eight concept/pattern references	Hidden	Optional (default: EL++)	Required
Navigation: navigation patterns	Hidden (inferred)	Optional (default: inferred)	Required
Navigation: vertex states	Hidden (ACTIVE_AND_INACTIVE)	Optional	Required
Navigation: sort controls	Hidden (sorted)	Optional	Required

6. Tier 1: Graph RAG Ready

6.1. Design Philosophy

Provide pre-materialized, human-readable views using server-side defaults. Every response returns resolved strings and labels. The caller never sees coordinate records or STAMP structures. Optimized for LLM context window consumption.

At Tier 1, component identity is simplified: callers provide a single UUID (any UUID from the component's PublicId is sufficient to identify it), and responses return a single canonical UUID per component. The full multi-UUID PublicId is exposed at Tiers 2 and 3.

6.2. Service Requirements

The Tier 1 service (working name: **IkeGraphRAG**) must provide the following RPCs:

6.2.1. GetDeviceView

Purpose: Retrieve a single concept with all semantics resolved to human-readable form.

Request must accept:

- A concept identifier (any single UUID from the component's PublicId)
- An optional timestamp (ISO 8601; defaults to latest)
- An optional language tag (e.g., "en-US"; defaults to server default)

Response must include:

- The concept's canonical UUID
- The preferred name (already resolved via language coordinate)
- A map of attribute names to string values (field meanings resolved to labels, values resolved to strings)
- A list of relationships, each with: human-readable relationship type, target UUID, target name (pre-resolved)
- The effective timestamp
- Categorical tags/classifications if applicable

6.2.2. GetSubgraph

Purpose: Retrieve a neighborhood of concepts for RAG context.

Request must accept:

- A starting concept identifier (any single UUID)
- Maximum traversal depth
- Optional relationship type filter (by human-readable type names)
- Optional timestamp

- Optional language tag

Response must include:

- A list of concept views (same structure as GetDeviceView responses)
- A list of edges with human-readable type, source, and target
- The effective timestamp

6.2.3. SearchConcepts

Purpose: Full-text search returning results ready for LLM consumption.

Request must accept:

- A search query string
- Optional result limit
- Optional timestamp
- Optional language tag

Response must include:

- Ranked results, each with: concept identifier (UUID), preferred name, relevance score, and a brief description or context snippet

6.2.4. GetGraphSnapshot

Purpose: Retrieve the state of a subgraph at a specific point in time. Supports regulatory use cases ("what did this look like on the approval date?").

Request must accept:

- A concept identifier or set of concept identifiers (UUIDs)
- A required timestamp (this is the one Tier 1 RPC where time is not optional — the whole point is temporal)
- Optional language tag

Response must include:

- Concept views as-of the specified timestamp
- Metadata indicating the snapshot time

6.3. Tier 1 Coordinate Contract

All coordinates are resolved server-side. The implementation must:

1. Maintain a well-known default coordinate configuration (the "Tier 1 defaults")
2. Allow the timestamp parameter to override `StampPositionRecord.time()` within the default

STAMP coordinate

3. Allow the language tag to select from a set of pre-configured `LanguageCoordinateRecord` instances
4. Use the default `LogicCoordinateRecord` and `NavigationCoordinateRecord` without exception
5. Document the Tier 1 defaults so that Tier 2/3 users can reproduce the same behavior explicitly

7. Tier 2: Concept-Aware

7.1. Design Philosophy

Expose the concept-oriented structure with optional coordinate control. Responses include both full `PublicIds` and pre-resolved labels (the "dual-return pattern," directly analogous to `PublicIdWithString`). Callers who don't need coordinate control get sensible defaults; callers who do can override any axis.

7.2. Service Requirements

The Tier 2 service (working name: `IkeKnowledgeGraph`) must provide the following RPCs:

7.2.1. `GetConceptWithSemantics`

Purpose: Retrieve a concept with its full semantic structure, including pattern references.

Request must accept:

- A concept `PublicId` (one or more UUIDs)
- An optional STAMP coordinate override
- An optional language coordinate override
- An optional logic coordinate override
- An optional navigation coordinate override

Response must include:

- The concept reference (`PublicId` + optional preferred label)
- A list of semantics, each with:
 - Semantic `PublicId`
 - Pattern reference (`PublicId` + label)
 - A list of fields, each with: field meaning (`PublicId` + label), field purpose (`PublicId` + label), datatype reference, and the typed value
 - STAMP information (status, time, author, module, path — each as `PublicId` + label)

7.2.2. `ResolveToText`

Purpose: Resolve a concept reference to its preferred text representation under a given coordinate

context.

Request must accept:

- A concept PublicId
- Optional coordinate overrides (any axis)

Response must include:

- The resolved text
- The description type used (e.g., preferred, fully qualified)
- The semantic reference for the description that was selected

7.2.3. GetSemanticsByPattern

Purpose: Retrieve all semantics matching a given pattern, optionally filtered to a specific concept.

Request must accept:

- A pattern PublicId
- An optional concept PublicId (to filter)
- Optional coordinate overrides

Response must include:

- A list of semantics (same structure as in GetConceptWithSemantics)
- Total count

7.2.4. QueryWithCoordinates

Purpose: Execute a query with fully explicit coordinate specification. This is the Tier 2 RPC that most closely approaches Tier 3 behavior.

Request must accept:

- A query expression (search terms, concept filter, pattern filter — specifics TBD)
- A complete or partial coordinate specification for each axis

Response must include:

- Query results with dual-return (PublicId + label) references
- The effective coordinates used (so the caller can see what defaults were applied)

7.2.5. GetVersionHistory

Purpose: Retrieve the version history of a concept.

Request must accept:

- A concept `PublicId`
- An optional path filter
- An optional time range

Response must include:

- An ordered list of version entries, each with: STAMP information and the semantics as-of that version

7.2.6. CompareVersions

Purpose: Compute a diff between two temporal states of the knowledge graph.

Request must accept:

- A "from" timestamp
- A "to" timestamp
- An optional scope (specific concepts, specific patterns, or the full graph)
- Optional coordinate overrides for the non-temporal axes

Response must include:

- A list of changes, each with: concept reference, change type (added/modified/retired), and a description of what changed

7.3. Tier 2 Coordinate Contract

Coordinate handling at Tier 2 must satisfy these requirements:

1. **Every coordinate axis is optional in requests.** The server applies a documented default for any axis not specified.
2. **Partial overrides are supported.** A caller may override STAMP without touching language, or override language dialect preference without specifying the full language coordinate.
3. **Responses include the effective coordinates.** When the server fills defaults, the response must indicate what was used, so the caller can learn the coordinate system incrementally.
4. **Coordinate structures at Tier 2 use PublicIds** (not nids) for all concept and pattern references, with optional labels for readability. The server translates between PublicIds and internal nids.
5. **The dual-return pattern applies everywhere.** Any concept reference in a Tier 2 response carries both a `PublicId` and a preferred label (analogous to `PublicIdWithString`).

7.4. Tier 2 Coordinate Override Structures

The Tier 2 coordinate override messages need not mirror the Java records field-for-field, but they must be *capable of expressing everything the Java records express*. The following specifies what each

override structure must support:

STAMP Coordinate Override:

- Allowed states (expressible as a set of status values, not a single status)
- Position time (ISO 8601 string; absent means latest)
- Position path (PublicId; absent means server default)
- Module inclusion set (list of PublicIds; absent means all)
- Module exclusion set (list of PublicIds; absent means none)
- Module priority list (ordered list of PublicIds; absent means server default)

Language Coordinate Override:

- Language concept (PublicId; absent means server default)
- Description pattern preference list (ordered PublicIds; absent means server default)
- Description type preference list (ordered PublicIds; absent means server default)
- Dialect pattern preference list (ordered PublicIds; absent means server default)
- Module preference for language (ordered PublicIds; absent means server default)

Logic Coordinate Override:

- Classifier (PublicId; absent means server default)
- Description logic profile (PublicId; absent means server default)
- Inferred axioms pattern (PublicId; absent means server default)
- Stated axioms pattern (PublicId; absent means server default)
- Concept member pattern (PublicId; absent means server default)
- Stated navigation pattern (PublicId; absent means server default)
- Inferred navigation pattern (PublicId; absent means server default)
- Root concept (PublicId; absent means server default)

Navigation Coordinate Override:

- Navigation pattern set (PublicIds; absent means server default)
- Vertex states (set of status values; absent means server default)
- Sort vertices flag (boolean; absent means server default)
- Sort pattern list (ordered PublicIds; absent means server default)

8. Tier 3: IKE Native

8.1. Design Philosophy

Provide raw access to concept/semantic/pattern/stamp primitives. No label pre-resolution. No default coordinates. The caller must specify the complete coordinate context for every query. Structures map faithfully to the Java records. This tier is for developers building their own IKE tooling, custom clients in other languages, or deep integrations.

8.2. Service Requirements

The Tier 3 service (working name: `IkePrimitives`) must provide the following RPCs:

8.2.1. GetConcept

Purpose: Retrieve a raw concept by PublicId.

Request must accept:

- A concept PublicId
- A complete STAMP coordinate

Response must include:

- The concept PublicId (no label resolution — Tier 3 callers resolve labels themselves via `ResolveWithCoordinates`)

8.2.2. GetSemantics (server streaming)

Purpose: Stream semantics matching specified criteria.

Request must accept:

- An optional concept PublicId filter
- An optional pattern PublicId filter
- A complete STAMP coordinate
- An optional limit

Response stream must yield:

- Individual semantics with raw concept/pattern references (PublicIds only, no labels)
- Full STAMP information per semantic
- Field values with unresolved concept references

8.2.3. ListPatterns (server streaming)

Purpose: Enumerate available patterns.

Request must accept:

- A complete STAMP coordinate
- An optional limit
- An optional continuation token for pagination

Response stream must yield:

- Pattern definitions: PublicId, meaning reference, purpose reference, ordered field definitions (each with index, meaning, purpose, datatype — all as PublicId references)

8.2.4. GetPattern

Purpose: Retrieve a single pattern definition.

Request must accept:

- A pattern PublicId
- A complete STAMP coordinate

Response: Same structure as a single element from ListPatterns.

8.2.5. ResolveWithCoordinates

Purpose: Manually resolve a concept to text using explicitly specified coordinates.

Request must accept:

- A concept PublicId
- A complete language coordinate
- A complete STAMP coordinate
- Optionally, a complete logic coordinate (if resolution depends on logic context)

Response must include:

- The resolved text
- The description type selected
- The semantic reference for the description

8.2.6. QueryBySTAMP (server streaming)

Purpose: Retrieve all components matching a STAMP query.

Request must accept:

- A complete STAMP coordinate
- An optional component type filter (concept, semantic, or pattern)

Response stream must yield:

- Components with PublicId, component type, and serialized data

8.2.7. StreamAllConcepts (server streaming)

Purpose: Iterate over all concepts in the system.

Request must accept:

- A complete STAMP coordinate
- An optional batch size hint

Response stream must yield:

- Concept PublicIds (no resolution)

8.3. Tier 3 Coordinate Contract

Coordinate handling at Tier 3 must satisfy these requirements:

1. **All coordinates are required.** A request missing any required coordinate must be rejected with a descriptive error, not silently defaulted.
2. **Coordinate structures must map faithfully to the Java records.** Each protobuf coordinate message must be capable of expressing exactly what the corresponding Java record expresses — no more, no less.
3. **The representation uses PublicIds, not nids.** Nids are an internal optimization. The gRPC layer translates between PublicIds (stable, portable, multi-UUID) and nids (fast, local). This translation is the server's responsibility.
4. **No label convenience fields.** Tier 3 responses contain only PublicIds. Label resolution is an explicit, separate operation via `ResolveWithCoordinates`.

8.4. Tier 3 Coordinate Structures

Unlike Tier 2 overrides (where every field is optional), Tier 3 coordinate messages have *required* fields. The following specifies the structure requirements:

STAMP Coordinate (all required):

- Allowed states (set of status values)
- Position time (epoch millis as int64, or a sentinel value for "latest")
- Position path (PublicId)
- Module inclusion set (list of PublicIds; empty list means "all modules")
- Module exclusion set (list of PublicIds; empty list means "no exclusions")
- Module priority list (ordered list of PublicIds; empty list means "no priority")

Language Coordinate (all required):

- Language concept (PublicId)
- Description pattern preference list (ordered PublicIds)
- Description type preference list (ordered PublicIds)
- Dialect pattern preference list (ordered PublicIds)
- Module preference for language (ordered PublicIds)

Logic Coordinate (all required):

- Classifier (PublicId)
- Description logic profile (PublicId)
- Inferred axioms pattern (PublicId)
- Stated axioms pattern (PublicId)
- Concept member pattern (PublicId)
- Stated navigation pattern (PublicId)
- Inferred navigation pattern (PublicId)
- Root concept (PublicId)

Navigation Coordinate (all required):

- Navigation pattern set (PublicIds)
- Vertex states (set of status values)
- Sort vertices (boolean)
- Sort pattern list (ordered PublicIds)

9. Design Principles

9.1. The Dual-Return Pattern

Tier 2 responses include both full PublicIds and pre-resolved labels wherever a concept reference appears — directly analogous to the `PublicIdWithString` pattern in the Java codebase. This serves as the bridge between tiers: pragmatic developers use labels, advanced developers use PublicIds, and the migration path is smooth. Tier 1 returns labels (and a single canonical UUID for linkability). Tier 3 returns PublicIds only.

9.2. Progressive Revelation of Complexity

The simple things must be simple (Tier 1). The complex things must be possible (Tier 3). There must be a clear, documented upgrade path between tiers. A developer who outgrows Tier 1 should be able to reproduce their Tier 1 results at Tier 2 by explicitly specifying the Tier 1 defaults.

9.3. Leverage Calculators, Don't Expose Them

The Java implementation uses calculator objects (e.g., `StampCalculatorWithCache`) for efficient, cached resolution. The gRPC API exposes calculator *results*, not calculator *interfaces*. Tiers 1 and 2 use calculators implicitly. Tier 3 allows manual resolution for cases where calculator assumptions don't fit, but still benefits from server-side calculator infrastructure.

9.4. FDA Compliance Awareness

The coordinate system directly supports regulatory use cases:

- STAMP position time enables "graph state as-of approval date" queries
- STAMP author and module tracking provide provenance and audit trail
- Immutable PublicIds ensure referential integrity across time
- Path coordinates support parallel development and release management

These capabilities should be surfaced as first-class features in Tier 1 (e.g., a regulatory snapshot request that takes a device identifier and a regulatory date) rather than requiring callers to understand STAMP coordinates to achieve compliance-grade queries.

10. Implementation Strategy

10.1. Phase 1: Launch with Tiers 1 and 2

Ship Tier 1 and Tier 2 together. Tier 1 provides the adoption ramp; Tier 2 provides the growth path. Tier 3 can be added incrementally based on demand.

Critically, Phase 1 must also deliver:

- **Documentation of the Tier 1 default coordinates.** Publish the exact `StampCoordinateRecord`, `LanguageCoordinateRecord`, `LogicCoordinateRecord`, and `NavigationCoordinateRecord` used by Tier 1 defaults.
- **A "Coordinate Cookbook"** with pre-built configurations for common use cases: US English clinical terms, multi-lingual with fallback, regulatory snapshot, stated vs. inferred navigation.

10.2. Phase 2: Tier 3 and Community Patterns

Based on Tier 2 usage data (which coordinate axes are people overriding?), finalize the Tier 3 contract and publish integration patterns for common toolchains (LangChain, Neo4j projection, Elasticsearch indexing).

10.3. Phase 3: Write Operations

This document specifies read-only operations. Write operations (creating concepts, authoring semantics, committing STAMP versions) represent a separate design effort that should build on the

coordinate infrastructure established here.

11. Performance Considerations

11.1. Caching

Java calculators handle internal caching. The gRPC layer should remain stateless. Tier 1 responses are highly cache-friendly because they use fixed default coordinates — identical requests produce identical results. Tier 2 and 3 responses vary by coordinate, so caching requires coordinate-aware cache keys.

11.2. Streaming vs. Unary

Use server streaming for: pattern iteration (Tier 3), large result sets, concept enumeration. Use unary RPCs for: single concept/semantic retrieval, bounded subgraph queries, resolution operations.

11.3. Batch Operations

Common Tier 1 operations (e.g., retrieving views for multiple concepts) should offer batch variants to reduce round-trip overhead. The batch request accepts multiple concept identifiers and shares a single coordinate context.

12. Open Questions for Reviewer Feedback

1. **Tier 2 partial overrides:** Should partial coordinate overrides be merged field-by-field with defaults, or should each coordinate axis be all-or-nothing? Field-by-field merging is more convenient but harder to reason about. All-or-nothing is simpler but less ergonomic.
2. **Time representation:** Should timestamps at the gRPC boundary be ISO 8601 strings, protobuf `Timestamp` messages, or raw int64 epoch millis? The Java records use `long` internally. ISO 8601 is human-readable. Protobuf `Timestamp` is idiomatic gRPC.
3. **StateSet representation:** Should the protobuf represent `StateSet` as a repeated enum, a bitmask, or named presets (ACTIVE_ONLY, ACTIVE_AND_INACTIVE)? The Java `StateSet` appears to use named constants.
4. **Error semantics:** When a coordinate specifies a `PublicId` that doesn't resolve to a valid component, should the server return a gRPC NOT_FOUND, INVALID_ARGUMENT, or a response with an error field? This matters especially for Tier 3 where coordinates are caller-specified.
5. **Module priority vs. module preference for language:** The STAMP coordinate has a `modulePriorityNidList` and the language coordinate has a `modulePreferenceNidListForLanguage`. Should the Tier 2 API surface these as a unified module preference, or keep them separate as the Java records do?
6. **PublicId wire encoding:** A `PublicId` carries one or more UUIDs. Should the protobuf encode each UUID as a string (human-readable, larger), as a pair of int64 (msb/lsb — compact, matches

`PublicId.forEach(LongConsumer))`, or as 16 bytes? The choice affects wire size, debuggability, and ease of use for Python/JS clients.

7. **Tier 1 identity simplification:** Tier 1 accepts a single UUID and returns a single canonical UUID per component for simplicity. Should Tier 1 responses also include the full PublicId (multiple UUIDs) as an optional field, to ease the transition to Tier 2? Or does that leak complexity into the "minutes to productivity" tier?

[1] IKE is architecturally based on the HL7 TINKAR (Terminology Knowledge Architecture) standard. References to coordinate records, STAMP versioning, and the self-describing concept model derive from this foundation.